

使用 CLI GDB 的偵錯示例

請檢查你的目標板是否已經準備好，並載入了 [get-started/blink](#) 示例程式碼，然後按照 [使用命令列偵錯](#) 中介紹的步驟組態和啟動偵錯程式，最後選擇讓應用程式在 `app_main()` 建立的斷點處停止運行

```
Temporary breakpoint 1, app_main () at /home/user-  
name/esp/blink/main/./blink.c:43  
43      xTaskCreate(&blink_task, "blink_task",  
configMINIMAL_STACK_SIZE, NULL, 5, NULL);  
(gdb)
```

本小節的示例

1. [瀏覽程式碼，查看堆疊和執行緒](#)
2. [設定和清除斷點](#)
3. [暫停和恢復應用程式的運行](#)
4. [單步執行程式碼](#)
5. [查看並設定記憶體](#)
6. [觀察和設定程序變數](#)
7. [設定條件斷點](#)
8. [偵錯 FreeRTOS 對象](#)

瀏覽程式碼，查看堆疊和執行緒

當看到 (gdb) 提示符的時候，應用程式已停止運行，LED 也停止閃爍。

要找到程式碼暫停的位置，輸入 `l` 或者 `list` 命令，偵錯程式會列印出暫停點（`blink.c` 程式碼檔案的第 43 行）附近的幾行程式碼

```
(gdb) l  
38      }  
39  }  
40  
41  void app_main()
```

```
42 {
43     xTaskCreate(&blink_task, "blink_task",
configMINIMAL_STACK_SIZE, NULL, 5, NULL);
44 }
(gdb)
```

也可以通過輸入 **l 30, 40** 等命令來查看特定行號範圍內的程式碼。

使用 **bt** 或者 **backtrace** 來查看哪些函數最終導致了此程式碼被呼叫：

```
(gdb) bt
#0  app_main () at /home/user-name/esp/blink/main/./blink.c:43
#1  0x400d057e in main_task (args=0x0) at /home/user-name/esp/esp-idf/components/esp32/./cpu_start.c:339
(gdb)
```

輸出的第 0 行表示應用程式暫停之前呼叫的最後一個函數，即我們之前列出的 `app_main()`。`app_main()` 又被位於 `cpu_start.c` 檔案第 339 行的 `main_task` 函數呼叫。

想查看 `cpu_start.c` 檔案中 `main_task` 函數的上下文，需要輸入 **frame N**，其中 $N = 1$ ，因為根據前面的輸出，`main_task` 位於 #1 下：

```
(gdb) frame 1
#1  0x400d057e in main_task (args=0x0) at /home/user-name/esp/esp-idf/components/esp32/./cpu_start.c:339
339     app_main();
(gdb)
```

輸入 **l** 將顯示一段名為 `app_main()` 的程式碼（在第 339 行）：

```
(gdb) l
334     ;
335 }
336 #endif
337 //Enable allocation in region where the startup stacks were
located.
338 heap_caps_enable_nonos_stack_heaps();
339 app_main();
340 vTaskDelete(NULL);
```

```
341 }
342
(gdb)
```

通過列印前面的一些行，你會看到我們一直在尋找的 `main_task` 函數：

```
(gdb) l 326, 341
326 static void main_task(void* args)
327 {
328     // Now that the application is about to start, disable boot
watchdogs
329     REG_CLR_BIT(TIMG_WDTCONFIG0_REG(0),
TIMG_WDT_FLASHBOOT_MOD_EN_S);
330     REG_CLR_BIT(RTC_CNTL_WDTCONFIG0_REG,
RTC_CNTL_WDT_FLASHBOOT_MOD_EN);
331 #if !CONFIG_FREERTOS_UNICORE
332     // Wait for FreeRTOS initialization to finish on APP CPU,
before replacing its startup stack
333     while (port_xSchedulerRunning[1] == 0) {
334         ;
335     }
336 #endif
337     //Enable allocation in region where the startup stacks were
located.
338     heap_caps_enable_nonos_stack_heaps();
339     app_main();
340     vTaskDelete(NULL);
341 }
(gdb)
```

如果要查看其他程式碼，可以輸入 `i threads` 命令，則會輸出目標板上運行的執行緒列表：

```
(gdb) i threads
Id   Target Id           Frame
  8   Thread 1073411336 (dport) 0x400d0848 in dport_access_init_core
(arg=<optimized out>)
    at /home/user-name/esp/esp-
idf/components/esp32/./dport_access.c:170
  7   Thread 1073408744 (ipc0) xQueueGenericReceive
(xQueue=0x3ffae694, pvBuffer=0x0, xTicksToWait=1644638200,
```

```
xJustPeeking=0) at /home/user-name/esp/esp-idf/components/freertos/./queue.c:1452
6   Thread 1073431096 (Tmr Svc) prvTimerTask (pvParameters=0x0)
   at /home/user-name/esp/esp-idf/components/freertos/./timers.c:445
5   Thread 1073410208 (ipc1 : Running) 0x4000bfea in ?? ()
4   Thread 1073432224 (dport) dport_access_init_core (arg=0x0)
   at /home/user-name/esp/esp-idf/components/esp32/./dport_access.c:150
3   Thread 1073413156 (IDLE) prvIdleTask (pvParameters=0x0)
   at /home/user-name/esp/esp-idf/components/freertos/./tasks.c:3282
2   Thread 1073413512 (IDLE) prvIdleTask (pvParameters=0x0)
   at /home/user-name/esp/esp-idf/components/freertos/./tasks.c:3282
* 1   Thread 1073411772 (main : Running) app_main () at /home/user-
name/esp/blink/main/./blink.c:43
(gdb)
```

執行緒列表顯示了每個執行緒最後一個被呼叫的函數以及所在的 C 原始檔名（如果存在的話）。

你可以通過輸入 **thread N** 進入特定的執行緒，其中 **N** 是執行緒 ID。我們進入 5 號執行緒來看一下它是如何工作的：

```
(gdb) thread 5
[Switching to thread 5 (Thread 1073410208)]
#0  0x4000bfea in ?? ()
(gdb)
```

然後查看回溯：

```
(gdb) bt
#0  0x4000bfea in ?? ()
#1  0x40083a85 in vPortCPUReleaseMutex (mux=<optimized out>) at
/home/user-name/esp/esp-idf/components/freertos/./port.c:415
#2  0x40083fc8 in vTaskSwitchContext () at /home/user-name/esp/esp-
idf/components/freertos/./tasks.c:2846
#3  0x4008532b in _frxt_dispatch ()
#4  0x4008395c in xPortStartScheduler () at /home/user-name/esp/esp-
idf/components/freertos/./port.c:222
#5  0x4000000c in ?? ()
#6  0x4000000c in ?? ()
```

```
#7 0x4000000c in ?? ()
#8 0x4000000c in ?? ()
(gdb)
```

如上所示，回溯可能會包含多個條目，方便查看直至目標停止運行的函數呼叫順序。如果找不到某個函數的原始碼檔案，將會使用問號 ?? 替代，這表示該函數是以二進制格式提供的。像 0x4000bfea 這樣的值是被呼叫函數所在的記憶體地址。

使用諸如 **bt**，**it threads**，**thread N** 和 **list** 命令可以瀏覽整個應用程式的程式碼。這給單步偵錯程式碼和設定斷點帶來很大的便利，下面將一一展開來討論。

設定和清除斷點

在偵錯時，我們希望能夠在關鍵的程式碼行停止應用程式，然後檢查特定的變數、記憶體、暫存器和外設的狀態。為此我們需要使用斷點，以便在特定某行程式碼處快速訪問和停止應用程式。

我們在控制 LED 狀態發生變化的兩處程式碼行分別設定一個斷點。基於以上程式碼列表，這兩處分別為第 33 和 36 程式碼行。使用命令 **break M** 設定斷點，其中 **M** 是具體的程式碼行：

```
(gdb) break 33
Breakpoint 2 at 0x400db6f6: file /home/user-
name/esp/blink/main/./blink.c, line 33.
(gdb) break 36
Breakpoint 3 at 0x400db704: file /home/user-
name/esp/blink/main/./blink.c, line 36.
```

輸入命令 **c**，處理器將運行並在斷點處停止。再次輸入 **c** 將使其再次運行，並在第二個斷點處停止，依此類推：

```
(gdb) c
Continuing.
Target halted. PRO_CPU: PC=0x400DB6F6 (active) APP_CPU:
PC=0x400D10D8

Breakpoint 2, blink_task (pvParameter=0x0) at /home/user-
name/esp/blink/main/./blink.c:33
33          gpio_set_level(BLINK_GPIO, 0);
```

```

(gdb) c
Continuing.
Target halted. PRO_CPU: PC=0x400DB6F8 (active)    APP_CPU:
PC=0x400D10D8
Target halted. PRO_CPU: PC=0x400DB704 (active)    APP_CPU:
PC=0x400D10D8

Breakpoint 3, blink_task (pvParameter=0x0) at /home/user-
name/esp/blink/main/./blink.c:36
36          gpio_set_level(BLINK_GPIO, 1);
(gdb)

```

只有在輸入命令 **c** 恢復程式執行後才能看到 LED 改變狀態。

查看已設定斷點的數量和位置，請使用命令 **info break**:

```

(gdb) info break
Num      Type          Disp Enb Address      What
2        breakpoint     keep y   0x400db6f6 in blink_task at
/home/user-name/esp/blink/main/./blink.c:33
          breakpoint already hit 1 time
3        breakpoint     keep y   0x400db704 in blink_task at
/home/user-name/esp/blink/main/./blink.c:36
          breakpoint already hit 1 time
(gdb)

```

請注意，斷點序號（在 Num 欄列出）從 2 開始，這是因為在偵錯程式啟動時執行 **thb app_main** 命令已經在 **app_main()** 函數處建立了第一個斷點。由於它是一個臨時斷點，已經被自動刪除，所以沒有被列出。

要刪除一個斷點，請輸入 **delete N** 命令（或者簡寫成 **d N**），其中 **N** 代表斷點序號:

```

(gdb) delete 1
No breakpoint number 1.
(gdb) delete 2
(gdb)

```

更多關於斷點的資訊，請參閱 [可用的斷點和觀察點](#) 和 [關於斷點的補充知識](#)。

暫停和恢復應用程式的運行

在偵錯時，可以恢復程式執行並輸入程式碼等待某個事件發生或者保持無限循環而不設定任何斷點。對於後者，想要返回呼試模式，可以通過輸入 **Ctrl+C** 手動中斷程序的運行。

在此之前，請刪除所有的斷點，然後輸入 **c** 恢復程式執行。接著輸入 **Ctrl+C**，應用程式會停止在某個隨機的位置，此時 **LED** 也將停止閃爍。偵錯程式會列印如下資訊：

```
(gdb) c
Continuing.
^CTarget halted. PRO_CPU: PC=0x400D0C00          APP_CPU:
PC=0x400D0C00 (active)
[New Thread 1073433352]

Program received signal SIGINT, Interrupt.
[Switching to Thread 1073413512]
0x400d0c00 in esp_vApplicationIdleHook () at /home/user-name/esp/esp-
idf/components/esp32/./freertos_hooks.c:52
52          asm("waiti 0");
(gdb)
```

在上圖所示的情況下，應用程式已經在 **freertos_hooks.c** 檔案的第 52 行暫停運行，現在你可以通過輸入 **c** 再次將其恢復運行或者進行如下所述的一些偵錯工作。

單步執行程式碼

我們還可以使用 **step** 和 **next** 命令（可以簡寫成 **s** 和 **n**）單步執行程式碼，這兩者之間的區別是執行 “**step**” 命令會進入呼叫的子程序內部，而執行 “**next**” 命令則會直接將子程序看成單個原始碼行，單步就能將其運行結束。

在繼續演示此功能之前，請使用前面介紹的 **break** 和 **delete** 命令，確保目前只在 **blink.c** 檔案的第 36 行設定了一個斷點：

```
(gdb) info break
Num      Type           Disp Enb Address      What
3        breakpoint      keep y   0x400db704 in blink_task at
/home/user-name/esp/blink/main/./blink.c:36
        breakpoint already hit 1 time
(gdb)
```

輸入 **c** 恢復程式執行然後等它在斷點處停止運行:

```
(gdb) c
Continuing.
Target halted. PRO_CPU: PC=0x400DB754 (active)    APP_CPU:
PC=0x400D1128

Breakpoint 3, blink_task (pvParameter=0x0) at /home/user-
name/esp/blink/main/./blink.c:36
36          gpio_set_level(BLINK_GPIO, 1);
(gdb)
```

然後輸入 **n** 多次，觀察偵錯程式是如何單步執行一行程式碼的:

```
(gdb) n
Target halted. PRO_CPU: PC=0x400DB756 (active)    APP_CPU:
PC=0x400D1128
Target halted. PRO_CPU: PC=0x400DB758 (active)    APP_CPU:
PC=0x400D1128
Target halted. PRO_CPU: PC=0x400DC04C (active)    APP_CPU:
PC=0x400D1128
Target halted. PRO_CPU: PC=0x400DB75B (active)    APP_CPU:
PC=0x400D1128
37          vTaskDelay(1000 / portTICK_PERIOD_MS);
(gdb) n
Target halted. PRO_CPU: PC=0x400DB75E (active)    APP_CPU:
PC=0x400D1128
Target halted. PRO_CPU: PC=0x400846FC (active)    APP_CPU:
PC=0x400D1128
Target halted. PRO_CPU: PC=0x400DB761 (active)    APP_CPU:
PC=0x400D1128
Target halted. PRO_CPU: PC=0x400DB746 (active)    APP_CPU:
PC=0x400D1128
33          gpio_set_level(BLINK_GPIO, 0);
(gdb)
```

如果你輸入 **s**，那麼偵錯程式將進入子程序:

```
(gdb) s
```



```
Target halted. PRO_CPU: PC=0x400DB748 (active)    APP_CPU:
PC=0x400D1128
Target halted. PRO_CPU: PC=0x400DB74B (active)    APP_CPU:
PC=0x400D1128
Target halted. PRO_CPU: PC=0x400DC04C (active)    APP_CPU:
PC=0x400D1128
Target halted. PRO_CPU: PC=0x400DC04F (active)    APP_CPU:
PC=0x400D1128
gpio_set_level (gpio_num=GPIO_NUM_4, level=0) at /home/user-
name/esp/esp-idf/components/esp_driver_gpio/src/gpio.c:183
183      GPIO_CHECK(GPIO_IS_VALID_OUTPUT_GPIO(gpio_num), "GPIO output
gpio_num error", ESP_ERR_INVALID_ARG);
(gdb)
```

上述例子中，偵錯程式進入 `gpio_set_level(BLINK_GPIO, 0)` 程式碼內部，同時程式碼窗口快速切換到 `gpio.c` 驅動檔案。

請參閱 [“next” 命令無法跳過子程序的原因](#) 文件以瞭解 `next` 命令的潛在侷限。

查看並設定記憶體

使用命令 `x` 可以顯示記憶體的內容，配合其餘參數還可以調整所顯示記憶體位置的格式和數量。運行 `help x` 可以查看更多相關細節。與 `x` 命令配合使用的命令是 `set`，它允許你將值寫入記憶體。

為了演示 `x` 和 `set` 的使用，我們將在記憶體地址 `0x3FF44004` 處讀取和寫入內容。該地址也是 `GPIO_OUT_REG` 暫存器的地址，可以用來控制（設定或者清除）某個 `GPIO` 的電平。

關於該暫存器的更多詳細資訊，請參閱 *ESP32 技術參考手冊* > *IO MUX 和 GPIO Matrix (GPIO, IO_MUX)* [\[PDF\]](#) 章節。

同樣在 `blink.c` 項目檔案中，在兩個 `gpio_set_level` 語句的後面各設定一個斷點。輸入兩次 `c` 命令後停止在斷點處，然後輸入 `x/1wx 0x3FF44004` 來顯示 `GPIO_OUT_REG` 暫存器的值：

```
(gdb) c
Continuing.
Target halted. PRO_CPU: PC=0x400DB75E (active)    APP_CPU:
PC=0x400D1128
Target halted. PRO_CPU: PC=0x400DB74E (active)    APP_CPU:
```

```
PC=0x400D1128
```

```
Breakpoint 2, blink_task (pvParameter=0x0) at /home/user-  
name/esp/blink/main/./blink.c:34
```

```
34          vTaskDelay(1000 / portTICK_PERIOD_MS);
```

```
(gdb) x /1wx 0x3FF44004
```

```
0x3ff44004: 0x00000000
```

```
(gdb) c
```

```
Continuing.
```

```
Target halted. PRO_CPU: PC=0x400DB751 (active)    APP_CPU:
```

```
PC=0x400D1128
```

```
Target halted. PRO_CPU: PC=0x400DB75B (active)    APP_CPU:
```

```
PC=0x400D1128
```

```
Breakpoint 3, blink_task (pvParameter=0x0) at /home/user-  
name/esp/blink/main/./blink.c:37
```

```
37          vTaskDelay(1000 / portTICK_PERIOD_MS);
```

```
(gdb) x /1wx 0x3FF44004
```

```
0x3ff44004: 0x00000010
```

```
(gdb)
```

如果閃爍的 LED 連接到了 GPIO4，那麼每次 LED 改變狀態時你會看到第 4 位元被翻轉：

```
0x3ff44004: 0x00000000
```

```
...
```

```
0x3ff44004: 0x00000010
```

現在，當 LED 熄滅時，與之對應地會顯示 0x3ff44004: 0x00000000，嘗試使用 set 命令向相同的記憶體地址寫入 0x00000010 來將該位元置高：

```
(gdb) x /1wx 0x3FF44004
```

```
0x3ff44004: 0x00000000
```

```
(gdb) set {unsigned int}0x3FF44004=0x000010
```

在輸入 **set {unsigned int}0x3FF44004=0x000010** 命令後，你會立即看到 LED 亮起。

觀察和設定程序變數

常見的偵錯任務是在程式執行期間檢查程序中某個變數的值，為了能夠演示這個功能，更新 `blink.c` 檔案，在 `blink_task` 函數的上面新增一個全域變數的聲明 `int i`，然後在 `while(1)` 裡新增 `i++`，這樣每次 LED 改變狀態的時候，變數 `i` 都會增加 1。

退出偵錯程式，這樣就不會與新程式碼混淆，然後重新建構並燒寫程式碼到 ESP32 中，接著重啟偵錯程式。注意，這裡不需要我們重啟 OpenOCD。

一旦程序停止運行，輸入命令 **watch i**:

```
(gdb) watch i
Hardware watchpoint 2: i
(gdb)
```

這會在所有變數 `i` 發生改變的程式碼處插入所謂的“觀察點”。現在輸入 **continue** 命令來恢復應用程式的運行並觀察它停止:

```
(gdb) c
Continuing.
Target halted. PRO_CPU: PC=0x400DB751 (active)   APP_CPU:
PC=0x400D0811
[New Thread 1073432196]

Program received signal SIGTRAP, Trace/breakpoint trap.
[Switching to Thread 1073432196]
0x400db751 in blink_task (pvParameter=0x0) at /home/user-
name/esp/blink/main/./blink.c:33
33          i++;
(gdb)
```

多次恢復程式執行後，變數 `i` 的值會增加，現在你可以輸入 **print i**（簡寫 **pi**）來查看當前 `i` 的值:

```
(gdb) p i
$1 = 3
(gdb)
```

要修改 `i` 的值，請使用 **set** 命令，如下所示（可以將其列印輸出來查看是否確已修改）:

```
(gdb) set var i = 0
(gdb) p i
$3 = 0
(gdb)
```

最多可以使用兩個觀察點，詳細資訊請參閱 [可用的斷點和觀察點](#)。

設定條件斷點

接下來的內容更為有趣，你可能想在一定條件滿足的情況下設定斷點。請先刪除已有的斷點，然後嘗試如下命令：

```
(gdb) break blink.c:34 if (i == 2)
Breakpoint 3 at 0x400db753: file /home/user-
name/esp/blink/main/./blink.c, line 34.
(gdb)
```

以上命令在 `blink.c` 檔案的 34 處設定了一個條件斷點，當 `i == 2` 條件滿足時，程序會停止運行。

如果當前 `i` 的值小於 2 並且程序被恢復運行，那麼 LED 就會循環閃爍，直到 `i == 2` 條件成立，最後程序停止在該處：

```
(gdb) set var i = 0
(gdb) c
Continuing.
Target halted. PRO_CPU: PC=0x400DB755 (active) APP_CPU:
PC=0x400D112C
Target halted. PRO_CPU: PC=0x400DB753 (active) APP_CPU:
PC=0x400D112C
Target halted. PRO_CPU: PC=0x400DB755 (active) APP_CPU:
PC=0x400D112C
Target halted. PRO_CPU: PC=0x400DB753 (active) APP_CPU:
PC=0x400D112C

Breakpoint 3, blink_task (pvParameter=0x0) at /home/user-
name/esp/blink/main/./blink.c:34
34          gpio_set_level(BLINK_GPIO, 0);
(gdb)
```

偵錯 FreeRTOS 對象

該部分內容或許可以幫助你偵錯 FreeRTOS 任務互動。

需要偵錯 FreeRTOS 任務互動的使用者可使用 GDB 命令 `freertos`。該命令並非 GDB 原生命令，而是來自於 Python 擴展模組 [freertos-gdb](#)，其包含一系列子命令：

```
(gdb) freertos
"freertos" 後面必須緊隨子命令的名稱
freertos 子命令如下：

freertos queue -- 列印當前佇列資訊
freertos semaphore -- 列印當前訊號量資訊
freertos task -- 列印當前任務及其狀態
freertos timer -- 列印當前定時器資訊
```

點選 <https://pypi.org/project/freertos-gdb> 連結瞭解此擴展模組的詳細資訊。

備註

ESP-IDF 在安裝 Python 包時會自動安裝 `freertos-gdb` Python 模組，詳情請參考 [第三步：設定工具](#)。

如果使用 `idf.py gdb` 命令運行 GDB，FreeRTOS 擴展會自動載入。也可以使用 GDB 內部命令 `python import freertos_gdb` 使能該模組。

請保證使用 Python 3.6 及以上版本，該版本具有 Python 共享庫。

獲得命令的幫助資訊

目前所介紹的都是些非常基礎的命令，目的在於讓你快速上手 JTAG 偵錯。如果想獲得特定命令的語法和功能相關的資訊，請在 (gdb) 提示符下輸入 `help` 和命令名：

```
(gdb) help next
Step program, proceeding through subroutine calls.
Usage: next [N]
Unlike "step", if the current source line calls a subroutine,
this command does not enter the subroutine, but instead steps over
the call, in effect treating it as a single source line.
(gdb)
```

只需輸入 **help** 命令，即可獲得高級命令列表，幫助你瞭解更多詳細資訊。此外，還可以參考一些 GDB 命令速查表，比

如 <https://darkdust.net/files/GDB%20Cheat%20Sheet.pdf>。雖然不是所有命令都適用於嵌入式環境，但還是會有所裨益。

結束偵錯會話

輸入命令 **q** 可以退出偵錯程式：

```
(gdb) q
A debugging session is active.

    Inferior 1 [Remote target] will be detached.

Quit anyway? (y or n) y
Detaching from program: /home/user-name/esp/blink/build/blink.elf,
Remote target
Ending remote debugging.
user-name@computer-name:~/esp/blink$
```